

challenge 2

2312300 曲恒睿

问题 1: `csw sscratch, sp` 与 `crrw s0, sscratch, x0` 的作用与目的

问题说明

在 `trapentry.S` 中的陷入入口部分，汇编代码中包含两条指令：

`csw sscratch, sp` 和 `crrw s0, sscratch, x0`。

它们的具体操作是什么？为什么需要这样设计？

答案分析

陷入发生时，CPU 需要立即保存当前的执行现场。然而此时我们并不知道当前栈指针 `sp` 是否安全（例如，如果陷入发生在用户态，用户栈不属于内核可写区域）。

因此，RISC-V 提供了 `sscratch` 这个特殊的寄存器，用作陷入时的临时缓冲。

这两条指令的作用如下：

- `csw sscratch, sp`：把当前的栈指针 `sp` 写入 `sscratch`，相当于把原来的栈顶地址暂存在一个安全的地方。
- `crrw s0, sscratch, x0`：从 `sscratch` 取出刚才保存的旧栈指针，放入 `s0`，并把 0 写回 `sscratch`。

这两个操作的目的是在陷入刚发生、尚未切换栈之前，先安全地保存旧栈指针。

后续 `SAVE_ALL` 宏会用到这个值来正确地把寄存器压入栈中。

如果陷入来自用户态，`sscratch` 事先存放了内核栈的地址，CPU 就能安全地切换到内核栈；

如果陷入来自内核态，`sscratch` 为空，则不会切换栈，只完成保存工作。

这套机制保证了陷入处理在任何情况下都能安全、无分支地执行。

问题 2: 为什么 `SAVE_ALL` 保存了 `stval`、`scause` 等 CSR，但 `RESTORE_ALL` 不恢复它们？

问题说明

在 `SAVE_ALL` 宏中，除了保存所有通用寄存器外，还保存了若干 CSR，例如 `sstatus`、`sepc`、`stval`、`scause`。

但在 `RESTORE_ALL` 宏中，只恢复了 `sstatus` 和 `sepc`，没有恢复 `stval`、`scause`。

为什么要保存却不恢复？这样做的意义是什么？

答案分析

这些 CSR 分为两类：

一类是影响 CPU 执行状态的寄存器，如 `sstatus` 和 `sepc`；

另一类是仅记录异常信息的寄存器，如 `stval` 和 `scause`。

`sstatus` 和 `sepc` 会直接影响程序返回后的执行环境，必须恢复，否则 CPU 无法正确回到被中断的位置。

而 `stval` 和 `scause` 只是为了让内核了解“发生了什么”，它们包含异常原因、错误地址等现场信息，仅用于中断处理函数读取分析。

恢复这些寄存器既没有意义，也可能破坏下一个陷阱的现场信息，因为硬件会在新的陷阱发生时自动重新写入。

因此，保存 `stval` 和 `scause` 的意义在于：
让上层的 C 函数能够访问这些信息，用于打印、调试和判断异常类型。
不恢复它们是为了保持陷阱现场的语义正确性。

实验总结

在陷入处理的最初阶段，`csrw sscratch, sp` 与 `csrrw s0, sscratch, x0` 负责暂存旧栈指针，为安全换栈和后续保存现场提供基础。

而 `SAVE_ALL` 保存的 CSR 可分为两类：

用于恢复执行状态的寄存器，和用于分析现场的寄存器（`stval`、`scause`）。

前者必须恢复以保证程序正确返回，后者仅供读取和诊断，不参与恢复过程。